

Memory-Aware Software Engineering Agents: Architectures, Mechanisms, and Open Gaps

Michael Banf, Johannes Kuhn, Slawomir Garcarz and Max Becker

Perelyn GmbH, Munich, Germany
michael.banf@perelyn.com

Abstract. AI-assisted software engineering has progressed from code completion to agentic systems operating across the full development lifecycle. Yet these agents remain fundamentally stateless, i.e. each task begins without memory of prior sessions, resolved bugs, or accumulated project understanding. We present a survey that synthesizes evidence from a feature-level analysis of ten production software engineering harnesses as well as a quantitative classification the current state of the art memory architecture for software engineering agents research. Our analysis reveals an episodic-temporal deficit, i.e. no production harness ships episodic memory or temporal versioning natively, whereas a rapidly growing ecosystem of MCP-based memory add-ons has emerged to fill the deficit. Yet, even these rarely implement temporal reasoning. Similarly, within the research community, while addressing episodic memory only a small fraction consider temporal mechanisms. At the same time, benchmarks confirm a factual-to-cognitive gap across production memory architectures. We argue that persistent, temporally grounded memory is the prerequisite for software engineering agents that learn alongside the teams they serve.

Keywords: Software Engineering Agents · Agentic Memory · Context Engineering · Temporal Reasoning · Knowledge Graphs

1 Introduction

Agentic AI systems now operate across the full software engineering lifecycle [1,2,3]. Whether resolving complex bugs through multi-file analysis, discovering architectural patterns, or coordinating code reviews across a team, these agents accumulate significant task-specific knowledge during each session. Yet that knowledge vanishes when the session ends: the next invocation begins from scratch, repeating investigations, rediscovering constraints, and re-learning project conventions.

The field has responded rapidly to the challenge: from approximately 25 papers in 2023 to over 200 in 2025 [5,6], with every major AI provider now shipping persistent memory. Interestingly, no production harness implements episodic memory or temporal versioning natively, yet a thriving ecosystem of community-built MCP memory add-ons has emerged, evidence that practitioners feel the deficit acutely. However fundamental properties, such as temporal versioning, team-shared memory, or decision provenance are only rarely addressed.

2 Memory in Production Software Engineering Harnesses

The CoALA taxonomy [4] maps human memory types onto agent components, i.e. *working* (context window), *semantic* (conventions, facts), *episodic* (past debugging experiences), *procedural* (workflows, tool usage). Thereby, working memory is well-served by modern context windows and semantic memory is partially addressed by context files [7]. However, episodic and procedural memory are almost entirely absent. A finer-grained functional taxonomy [5] distinguishes *factual* (what the agent knows) from *experiential* (what the agent has learned from doing), and *working* memory (active context management) with all three being critical for software engineering agents operating over long project horizons. From this perspective, we analyzed the memory mechanisms of ten widely-used software engineering agent harnesses. Table 1 summarizes the comparison across six key dimensions derived from the CoALA taxonomy.

All ten harnesses support human-written instruction files, representing declarative semantic memory, with widespread adoption [7], yet little resolution improvement [8]. Context engineering [?] formalizes the approach, but all implementations remain manually authored without temporal metadata. Five harnesses generate memories automatically (Claude Code, Codex CLI, Copilot, Windsurf, Gemini CLI), but none distinguishes factual from experiential knowledge. Context compaction triggers vary with respect to window utilization, however implementations are lossy, i.e. technical details, intermediate reasoning steps, and failed-approach information are discarded. The quality of compaction determines whether memory helps or hinders [21]. No production software engineering agent supports temporal versioning, i.e. tracking when knowledge was valid, when it changed, or when it became stale. Neither do they support episodic memory, i.e. structured records of past debugging sessions, resolution trajectories, or decision rationale. Exactly, those memory capabilities most critical for software engineering agents that learn over time are precisely those that no tool implements.

2.1 The community response: MCP memory add-ons

The gap between what harnesses ship and what practitioners need has spawned a rapidly growing ecosystem of community-built memory extensions, primarily via the Model Context Protocol (MCP) [38]. Our survey identifies over 40 MCP-based memory servers (Table 1). The ecosystem spans every CoALA memory type, however with stark imbalances, i.e. semantic memory is near-universal (88% of servers), episodic recall is emerging (30%), while temporal versioning (5% bitemporal) and procedural memory (10%) remain nearly absent.

Among the most adopted, *claude-mem* [40] hooks into Claude Code lifecycle events for session capture. *AgentMemory* [39] provides hybrid BM25+vector+KG retrieval and *Graphiti/Zep* [12] remains the only production system with true bi-temporal validity intervals. Software engineering specific servers include *Codebase-Memory* [36] (Tree-Sitter KG across 155 languages), *Engram-MCP* [41] (git-branch-aware session handoffs), and *Hindsight* [42] (failed attempt replay).

	<i>Semantic</i>	<i>Episodic</i>	<i>Temporal</i>	<i>Procedural</i>	<i>Collab.</i>	<i>Code-Str.</i>
<i>Production Agents</i>						
 Claude Code	✓	✗	✗	✗	✗	✗
 Gemini CLI	✓	◇	◇	◇	✗	✗
 Codex CLI	✓	✗	◇	✗	✗	✗
 Copilot	✓	✗	◇	✗	✗	✗
 Cursor	✓	✗	✗	✗	✗	✗
 OpenCode	✓	✗	✗	✗	✗	✗
 Windsurf	✓	✗	✗	✗	✗	✗
 Devin	✓	✗	✗	✗	◇	✗
 Aider	◇	✗	✗	✗	✗	✗
 OpenHands	◇	✗	✗	✗	✗	✗
<i>Selected MCP Community Add-ons</i>						
 claude-mem	✓	✓	✗	◇	✗	✗
 MemO	✓	◇	✗	✗	✓	✗
 Graphiti / Zep	✓	✓	✓	✗	✗	✗
 Cognee	✓	✗	✗	◇	✗	✓
 AgentMemory	✓	✓	✗	✓	✗	✗
 Letta / MemGPT	✓	✓	✗	✓	✗	✗
 Codebase-Memory	✓	✗	✗	✗	✗	✓
 Engram-MCP	✗	✓	◇	✗	✗	✗
 Ourmem	✓	✗	✗	✗	✓	✗
 Hindsight	✗	✓	✗	✗	✗	✗
<i>Ecosystem coverage</i>	<i>88%</i>	<i>30%</i>	<i>13%</i>	<i>10%</i>	<i>13%</i>	<i>16%</i>

 Supported
  Partial
  Absent

Fig. 1. Memory type coverage across production SE agents (built-in) and representative MCP community add-ons. Temporal and procedural memory are the critical gaps.

Notably, nine of ten major coding agents now support MCP (only Aider lacks native integration), making these add-ons agent-agnostic. Built-in memory sophistication varies widely. For example, Gemini CLI offers a four-tier hierarchy with automatic skills extraction (episodic-to-procedural conversion), Copilot performs just-in-time codebase verification of stored facts, while Cursor has removed its auto-memory feature, suggesting that naive auto-capture without proper architecture degrades quality.

3 Memory Architectures: From Extraction to Graphs

Our classification of all ICRL 2026 MemAgents workshop papers [6] reveals at least nine distinct method categories: retrieval-augmented (20%), graph-based (11%), hierarchical consolidation (10%), experience replay (9%), extraction-based (9%), KV-cache compression (7%), embedding-based (6%), RL-based management (6%), and attention-based (4%). Among these, three dedicated memory systems have reached production maturity and been formally benchmarked head-to-head on the LoCoMo-Plus cognitive memory evaluation [9], i.e. extraction-based, self-managing, and graph-based temporal. All three build on the standard retrieval-augmented pipeline [5] but differ in how they organize, consolidate, and temporally ground stored memories.

3.1 Extraction-Based Memory

Extraction-based architectures, such as Mem0 [10], extract atomic facts from conversations and store them as vector-indexed entries for similarity based querying. Extraction-based systems excel at *factual recall*. For software engineering tasks requiring explicit knowledge retrieval, this paradigm is well-suited. However, extraction is lossy by design, e.g. the process of converting debugging sessions into atomic facts discards causal chains, temporal context, and reasoning paths [13].

3.2 Self-Managing Memory

Self-Managing Memory, such as Letta [11], treats the LLM itself as an operating system managing its own memory via explicit read/write/search operations over a hierarchical store. The agent decides what to remember and how to organize it. Self-managing memory excels at preserving *behavioral continuity*, i.e. reasoning on goals, recurring patterns, and evolving state. For software engineering agents that must maintain an evolving picture of project architecture, track long-running refactoring goals, or remember what patterns emerged during debugging, this is the strongest fit, although less transparent than explicit stores: the agent’s internal state cannot easily be inspected, audited, or corrected by developers.

3.3 Graph-Based Temporal Memory

Graph-Based temporal approaches, such as Graphiti [12], can distinguish *event time* (when something happened) from *ingestion time* (when the system learned it), with every edge carrying explicit validity intervals. This *bi-temporal* approach is unique among production memory systems. Graph-based memory excels at *causal reasoning*. For software engineering tasks requiring understanding of *why* a decision was made, how events connect across time, or what caused a regression, temporal graphs provide the most natural representation. However, Graph construction adds latency and complexity.

3.4 Hybrid and Emerging Approaches

The LoCoMo-Plus results make a compelling case for *hybrid architectures*, since no single paradigm masters all competency types [9]. The analysis confirms that the systems capture genuinely different aspects of the memory problem. To this end several emerging directions point toward future software engineering relevant architectures. Reinforcement learning based memory management, such as Memory-R1 [26] and AgeMem [27] replace heuristic memory policies with trained ones. The learned policies discover non-obvious strategies such as pre-emptive summarization before the context window fills, suggesting a path toward adaptive memory management tuned to software engineering specific reward signals such as issue resolution rate, code quality metrics. Multi-graph memory, as in MAGMA [23], represents each memory item across orthogonal semantic, temporal, causal, and entity graphs, with retrieval formulated as policy-guided traversal. This decomposition offers fine-grained control particularly relevant for software engineering: dependency relationships (entity graph), decision provenance (causal graph), API validity (temporal graph), and conceptual similarity (semantic graph) can be queried independently. Hierarchical consolidation provides a formal theory of hierarchical memory [?] defines three operators, i.e. extraction, coarsening, and traversal. This identifies a self-sufficiency spectrum for representative functions that constrains viable retrieval strategies. TiMem [24] implements this via a Temporal Memory Tree. For software engineering agents processing long execution traces, hierarchical consolidation could be essential. Hybrid local/cloud architectures are also emerging: LightMem [43] delegates memory operations (routing, filtering) to small local models while reserving large models for consolidation, and HyMem [44] achieves comparable performance at reduced cost via dynamic two-tier retrieval. Since the majority of MCP memory servers support fully local operation and frameworks like Mem0, Letta, and Cogneerun on Ollama, sovereign deployments are feasible, though reliable MCP tool calling requires 14B+ parameter models [35,17].

4 Memory Mechanisms for Software Engineering

Beyond general-purpose memory architectures, several mechanisms have been developed specifically for or with direct relevance to software engineering agents.

4.1 Versioned Context Management and Decision Provenance

The Git Context Controller (GCC) [14] introduces Git-inspired operations for agent memory, addressing the *intra-session* memory problem: managing working memory within a single task execution. The Lore protocol [15] addresses the complementary *inter-session* problem: it repurposes git commit messages as structured decision records carrying constraints, rejected alternatives, agent directives, and verification metadata via native git trailers, introducing queryable decision provenance. The practical need for decision provenance is acute: each

additional deviation in a mutating action reduces success odds [30], with errors growing as context length increases and agents drift from stale constraints, such as deprecated APIs [28,29].

4.2 Experiential and Cross-Session Learning

SWE Context Bench [21] benchmark evaluates whether agents benefit from prior experience. The critical finding: *summarized* experience improves resolution, while *unfiltered* experience hurts. Experiential Co-Learning [16] mines experiences from historical trajectories and injects them into future task execution. Experiential Reflective Learning [17] reflects on task trajectories to generate transferable heuristics, i.e. actionable lessons that generalize across tasks. Live-SWE-agent [18], rather than remembering past experiences, self-creates new tools during execution, representing procedural memory through self-modification. Structurally Aligned Subtask-Level Memory [19] decomposes software engineering problem-solving into subtasks and aligns memory storage/retrieval at the subtask level rather than the whole-instance level. MemGrad [20] transforms batches of behavioral feedback into a retrospective, prospective dual memory, i.e. retrospective memory captures recurring failure modes, while prospective memory encodes gradient-derived strategies for future reasoning. Procedural memory, such as (Mem^P) [37] distills past agent trajectories into both fine-grained step-by-step instructions and higher-level script-like abstractions, creating learnable, updatable procedural memory. Procedural knowledge built from a stronger model transfers effectively to weaker ones. Strategy-level memory, such as (ReasoningBank) [?], rather than storing raw trajectories, distills generalizable reasoning strategies from self-judged successes and failures. Finally, persistent note-taking [?] introduces a cross-session note-taking system where agents record hindsight notes for compilation errors, runtime exceptions, and unproductive strategies. Cognitive threshold for experiential memory, such as WebCoach [35], finds a striking capacity threshold: 7B models do not benefit from experiential memory, while 32B+ models show pronounced gains. Self-generated experiences outperform externally seeded ones [17]. For software engineering agents, this implies that experiential memory requires both sufficient model capacity to reason over retrieved experience and careful curation to avoid context pollution.

4.3 Knowledge Graphs for Software Engineering Agent Memory

A comprehensive survey on graph-based agent memory [22] identifies four lifecycle phases: extraction (raw data $\rightarrow$ graph nodes/edges), storage (organization and indexing), retrieval (query-adaptive traversal), and evolution (update, consolidate, forget). For software engineering agents, graph memory offers three specific advantages. First, they naturally implement *dependency tracking* via relationships for inherently graph-structured code repositories, including files import modules, classes inherit from others, functions call functions [36]. Second, they provide *decision provenance*. Architectural Decision Records, code review discussions, and commit rationale form a causal graph of project decisions. Graph-based memory

can preserve the *why* behind code structure, enabling agents to understand constraints before proposing changes. Third, they lay a foundation for monitoring *temporal evolution*. For example, Graphiti’s bi-temporal model (event time vs. ingestion time with validity intervals) maps naturally onto software evolution, such as an invalid API, a recently adopted coding convention or deprecated dependency. GAM [25] demonstrates the value of hierarchical graph memory showing significant improvements over extraction-based approaches by decoupling episodic from semantic consolidation via an event progression graph.

5 Collaborative and Team Memory

Software engineering is inherently collaborative, yet memory research exhibits a strong single-agent bias [6]. To this end, Collaborative Memory [31] is the most complete framework for multi-user memory sharing with dynamic access control. It maintains two tiers: *private memory* (visible only to the originating user) and *shared memory* (selectively shared fragments), where each fragment carries immutable provenance attributes (contributing agents, accessed resources, timestamps). This architecture maps naturally onto SE teams: private developer context (local experiments, debugging notes) vs. shared project knowledge (coding conventions, architectural decisions, resolved issues). However, e.g. MINJA [32] demonstrates that shared memory agents are vulnerable to query-only injection attacks. For software engineering agents with access to codebases and deployment pipelines, memory poisoning is a critical supply-chain risk. SSGM [33] and MemArchitect [34] propose governance middleware for evolving memory, while the mnemonic sovereignty survey [?] addresses EU AI Act compliance. Yet none targets software engineering-specific governance needs: access control aligned with repository permissions, memory provenance linked to code review approvals, or temporal policies tied to release cycles.

6 Discussion: Open Gaps

Synthesizing our analysis, we identify several open gaps specific to memory for software engineering agents as inspirations for future research. Software engineering agents discard resolution trajectories after each session. Early research solutions exist (§4), but no production harness or community add-on integrates cross-session learning. No production software engineering agent tracks when knowledge was valid or became stale. Among community add-ons, only a tiny fraction implement bi-temporal validity intervals. No software engineering specific temporal memory exists. Also, there exists only one software engineering specific temporal benchmark. Convention retention, experience transfer, staleness detection, and decision provenance remain unevaluated. Collaborative memory is rarely addressed, yet software engineering teams need shared decisions, private debugging notes, and repository-aligned access control. Research on injection success on shared memory agents show that lack memory governance and security, with GDPR-compliant deletion remains unaddressed. Software engineering craft

knowledge is overwhelmingly procedural, yet procedural memory is the rarest type in both research and the community ecosystem. The MCP ecosystem provides a composable memory architecture where each server addresses a different CoALA memory type. Yet no orchestration layer decides which memory to consult when, resolves conflicts between overlapping stores, or manages lifecycle across layers. Assembling the full stack remains a manual integration challenge. Closing these gaps will require a coordinated effort and a clear vision of transforming software engineering agents from amnesic tools into *learning collaborators* that grow alongside the software they help build.

References

1. Becker, M.: Der KI-gestützte Software-Entwicklungsprozess. BVMW Online-Impuls, Perelyn GmbH (April 2026)
2. Liu, J., et al.: Large Language Model-Based Agents for Software Engineering: A Survey. In: ACM TOSEM (2024)
3. Jimenez, C.E., et al.: SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In: ICLR 2024
4. Sumers, T.R., Yao, S., Narasimhan, K., Griffiths, T.L.: Cognitive Architectures for Language Agents. In: TMLR (2024)
5. Hu, Y., et al.: Memory in the Age of AI Agents: A Survey — Forms, Functions and Dynamics. arXiv:2512.13564 (2025)
6. MemAgents: Memory for LLM-Based Agentic Systems. ICLR 2026 Workshop
7. Chatlatanagulchai, W., et al.: Agent READMEs: An Empirical Study of Context Files for Agentic Coding. arXiv:2511.12884 (2025)
8. Gloaguen, J., et al.: Evaluating AGENTS.md. MemAgents Workshop, ICLR 2026
9. Gurawaliya, H., et al.: Agentic Memory Realized: From Stateless Models to Cognitive Continuity. appliedAI Initiative (2026)
10. Chhikara, P., et al.: Mem0: Production-Ready Long-Term Memory. ECAI 2025
11. Packer, C., et al.: MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560 (2023)
12. Rasmussen, P., et al.: Zep: Temporal Knowledge Graph for Agent Memory. arXiv:2501.13956 (2025)
13. Diagnosing Retrieval vs. Utilization Bottlenecks in LLM Agent Memory. MemAgents Workshop, ICLR 2026
14. Wu, J., et al.: Git Context Controller. arXiv:2508.00031 (2025)
15. Stetsenko, I.: Lore: Git Commits as Structured Knowledge Protocol. arXiv:2603.15566 (2026)
16. Qian, C., et al.: Experiential Co-Learning of Software-Developing Agents. In: ACL 2024
17. ERL: Experiential Reflective Learning for Self-Improving LLM Agents. MemAgents Workshop, ICLR 2026
18. Xia, C.S., et al.: Live-SWE-agent: Can Software Engineering Agents Self-Evolve on the Fly? arXiv:2511.13646 (2025)
19. Subtask-Level Memory for SE Agents. arXiv:2602.21611 (2026)
20. Natekar, P., et al.: MemGrad: Memory-Guided Agentic SE Optimization. MemAgents Workshop, ICLR 2026
21. Zhu, J., et al.: SWE Context Bench: A Benchmark for Context Learning in Coding. arXiv:2602.08316 (2026)
22. Yang, C., et al.: Graph-based Agent Memory: A Survey. arXiv:2602.05665 (2026)
23. MAGMA: Multi-Graph Agentic Memory Architecture. arXiv:2601.03236 (2026)
24. TiMem: Temporal-Hierarchical Memory Consolidation. arXiv:2601.02845 (2026)
25. Wu, Z., et al.: GAM: Hierarchical Graph Memory for LLM-based Agents. In: ICLR 2026
26. Memory-R1: RL Framework with Memory Manager for Agent Memory. arXiv (2025)
27. AgeMem: Agentic Memory with Unified LTM/STM Management via RL. arXiv (2026)
28. Wang, C., et al.: Deprecated API Usage in LLM Code Completion. ICSE 2025
29. Ashik, A.N., et al.: Knowledge Conflicts from Evolving APIs. arXiv:2604.09515 (2026)
30. SABER: Safeguarding Mutating Steps in LLM Agents. MemAgents Workshop, ICLR 2026
31. Rezazadeh, A., et al.: Collaborative Memory for LLM Agents. arXiv:2505.18279 (2025)
32. Zeng, H., et al.: MINJA: Memory Injection Attacks on LLM Agents. MemAgents Workshop, ICLR 2026
33. Lam, C., et al.: SSGM: Governing Evolving Memory in LLM Agents. arXiv:2603.11768 (2026)
34. Kumar, L.S., et al.: MemArchitect: Policy-Driven Memory Governance. arXiv:2603.18330 (2026)
35. WebCoach: Self-Evolving Web Agents with Cross-Session Memory. MemAgents Workshop, ICLR 2026

- 36. Vogel, M., et al.: Codebase-Memory: Tree-Sitter KG via MCP. arXiv:2603.27277 (2026)
- 37. Fang, R., et al.: MemP: Exploring Agent Procedural Memory. arXiv:2508.06433 (2025)
- 38. Anthropic: Model Context Protocol. <https://modelcontextprotocol.io> (2026)
- 39. agentmemory: Persistent Memory MCP Server. GitHub (2026)
- 40. claude-mem: Lifecycle-Hook Memory for AI Coding Agents. GitHub (2026)
- 41. edg-l: Engram-MCP — Git Branch-Aware Session Handoffs. GitHub (2026)
- 42. Shazil, S.: Hindsight — Attempt Memory for Coding Agents. GitHub (2026)
- 43. LightMem: SLMs for Online Memory Management. arXiv:2604.07798 (2026)
- 44. HyMem: Dynamic Dual-Granular Retrieval. arXiv:2602.13933 (2026)